

Python AI Bootcamp

Level 1: Python Foundations

Clear explanations + complete runnable code for every topic
Author: Venkatesan Vetrimurasu | YouTube: Venkat's Tech Lab

This level teaches the essential Python skills you need before starting Machine Learning and Deep Learning. Each topic starts with a plain-English explanation of the idea, then shows a complete working example, and ends with a small practice challenge.

How to use this guide: Create a folder, copy each sample into its own .py file, run it, then change it and break it on purpose. Fixing what you break is the fastest way to learn.

Before We Start: What is Python?

Python is a programming language. A programming language is a set of instructions that a computer can follow. We write these instructions in a text file (called a script or program). When we run the file, the computer executes the instructions one by one.

Python is popular for AI because it is readable (looks almost like English), has a huge collection of free libraries (NumPy, Pandas, PyTorch, etc.), and lets you try ideas quickly. Almost every modern AI tutorial and research paper uses Python.

Core ideas you will use every day

- **Variable** — a name that stores a value (like a labeled box).
- **Type** — what kind of value it is: number, text, true/false, list, etc.
- **Function** — a reusable block of code that does one job.
- **Loop** — repeat an action many times.
- **Condition** — do something only if a rule is true.
- **File** — permanent storage on disk (CSV, JSON, images, models).

Level 1 trains these ideas with small projects. After this level you will be ready for NumPy, Pandas, and real ML code.

Table of Contents

1. Variables, types, f-strings — Unit Converter
2. Lists, dicts, sets, tuples — Word Counter
3. Loops and comprehensions — Filter a Dataset
4. Functions, *args, **kwargs, lambdas — Calculator
5. File I/O + pathlib — Contact Manager
6. Error handling and custom exceptions
7. OOP and dataclasses — Dataset class
8. Modules, packages, venv, pip / uv
9. Reproducibility: seeds and pinned environments
10. Type hints with mypy
11. Pydantic: validation and settings
12. Generators, iterators, decorators
13. Context managers (with statement)
14. collections (defaultdict, deque, NamedTuple)
15. requests and REST APIs
16. Async with asyncio and httpx
17. Simple CLI with argparse

1. Variables, Types, f-strings — Unit Converter

The basic idea

A **variable** is a name that holds a value. You create one with the equals sign: `age = 25`. Python figures out the type automatically. Common types are: `int` (whole number), `float` (decimal), `str` (text), and `bool` (`True/False`).

An **f-string** is a text string that can insert variable values. Write an `f` before the quotes and put expressions inside curly braces: `f"Hello, {name}"`. This is the cleanest way to print results and logs.

Why this matters for AI

You constantly convert units (pixels to meters, Celsius to Kelvin), format training logs, and print metrics. Clear variables and f-strings keep experiments readable.

Complete sample

```
# 01_unit_converter.py
def convert_length(value: float, from_unit: str, to_unit: str) -> float:
    # Convert everything through meters first
    to_meters = {
        "m": 1.0, "km": 1000.0, "cm": 0.01, "mm": 0.001,
        "mile": 1609.34, "yard": 0.9144, "foot": 0.3048, "inch": 0.0254,
    }
    if from_unit not in to_meters or to_unit not in to_meters:
        raise ValueError(f"Unsupported unit. Supported: {list(to_meters.keys())}")
    meters = value * to_meters[from_unit]
    return meters / to_meters[to_unit]

value, from_u, to_u = 5.0, "km", "mile"
converted = convert_length(value, from_u, to_u)
print(f"{value} {from_u} = {converted:.4f} {to_u}")
# Output: 5.0 km = 3.1069 mile
```

Practice: Add temperature conversion ($^{\circ}\text{C} \leftrightarrow ^{\circ}\text{F} \leftrightarrow \text{K}$). Then add weight ($\text{kg} \leftrightarrow \text{lb}$).

2. Lists, Dicts, Sets, Tuples — Word Counter

The basic idea

Python has four main ways to group values:

- **list** `[1, 2, 3]` — ordered, you can change it (add/remove items).
- **tuple** `(1, 2, 3)` — ordered, but cannot be changed after creation.
- **dict** `{"name": "Alice"}` — maps keys to values (like a lookup table).
- **set** `{1, 2, 3}` — unordered collection of unique items (no duplicates).

Counter (from the `collections` module) is a special dict that counts how many times each item appears. It is perfect for word frequencies.

Why this matters for AI

Datasets are lists of records. Feature dictionaries, vocabulary sets, and configuration tuples appear in every project. Counting tokens or class labels is one of the first steps in NLP and classification.

Complete sample

```
# 02_word_counter.py
from collections import Counter

text = """Python is great. Python is powerful.
Learning Python with hands-on projects is the best way."""

words = text.lower().replace(".", "").replace(",", "").split()
word_counts = Counter(words)

print("Word frequencies:")
for word, count in word_counts.most_common():
    print(f"{word:15} → {count}")

unique_words = set(words)
print(f"\nTotal unique words: {len(unique_words)}")
print(f"Unique words: {sorted(unique_words)}")
```

Practice: Count pairs of consecutive words (bigrams). This is the start of simple language models.

3. Loops and Comprehensions — Filter a Dataset

The basic idea

A **loop** repeats code. The most common is `for item in collection:`. You can also use `while condition:` when you do not know the number of steps in advance.

A **list comprehension** is a short way to build a new list from an old one in one line: `[x*2 for x in numbers if x > 0]`. It means: take each `x`, keep it only if `x > 0`, and put `x*2` into the new list. Dict and set comprehensions work the same way with curly braces.

Why this matters for AI

Cleaning data, selecting rows, and transforming features are mostly filtering and mapping. Comprehensions keep this code short and readable.

Complete sample

```
# 03_filter_dataset.py
students = [
    {"name": "Alice", "age": 22, "score": 88},
    {"name": "Bob", "age": 19, "score": 72},
    {"name": "Charlie", "age": 25, "score": 95},
    {"name": "Diana", "age": 20, "score": 61},
    {"name": "Eve", "age": 23, "score": 79},
]

passed = [s["name"] for s in students if s["score"] >= 75]
print("Passed:", passed)

high_scores = {s["name"]: s["score"] for s in students if s["score"] >= 80}
print("High scores:", high_scores)

older = [s for s in students if s["age"] > 21]
avg = sum(s["score"] for s in older) / len(older)
print(f"Average score (age > 21): {avg:.1f}")
```

Practice: Compute the average using `statistics.mean` and a generator expression.

4. Functions, *args, **kwargs, Lambdas — Calculator

The basic idea

A **function** is a named block of code you can call many times. You define it with `def name(parameters):` and optionally return a value.

*args collects extra positional arguments into a tuple. **kwargs collects extra keyword arguments into a dict. Together they let you write flexible functions that accept any number of inputs.

A **lambda** is a tiny anonymous function written in one line: `square = lambda x: x**2`. Useful for short operations passed to sorted, map, or filter.

Why this matters for AI

Training loops, metric calculators, and data transforms are all functions. Libraries like PyTorch and scikit-learn use *args/**kwargs heavily in their APIs.

Complete sample

```
# 04_calculator.py
def calculate(operation, *args, **kwargs):
    if not args:
        raise ValueError("At least one number required")
    precision = kwargs.get("precision", 2)
    verbose = kwargs.get("verbose", False)

    if operation == "add":
        result = sum(args)
    elif operation == "multiply":
        result = 1
        for num in args:
            result *= num
    elif operation == "average":
        result = sum(args) / len(args)
    else:
        raise ValueError(f"Unknown operation: {operation}")

    result = round(result, precision)
    if verbose:
        print(f"Operation: {operation} | Numbers: {args} → Result: {result}")
    return result

square = lambda x: x ** 2
is_even = lambda x: x % 2 == 0

print(calculate("add", 10, 20, 30, precision=0, verbose=True))
print(calculate("multiply", 2, 3, 4, 5))
print(square(7))
print(list(filter(is_even, range(10))))
```

***Practice:** Add a 'power' operation. Write a function that takes any number of lists and returns element-wise averages.*

5. File I/O + pathlib — Contact Manager

The basic idea

Programs need to save and load data. **File I/O** (Input/Output) means reading from and writing to files on disk.

Always open files with a `with` statement so the file is closed automatically: `with open(path, "r") as f:`. Use `pathlib.Path` for paths — it works the same on Windows, Mac, and Linux.

Common formats: **TXT** (plain text), **CSV** (tables, like Excel), **JSON** (structured data, very common for configs and APIs).

Why this matters for AI

Every project starts by loading data. JSON is the standard for configs and API responses. CSV is the everyday format for tabular datasets.

Complete sample

```
# 05_contact_manager.py
import json, csv
from pathlib import Path

CONTACTS_FILE = Path("contacts.json")
```

```
def load_contacts():
    if CONTACTS_FILE.exists():
        with open(CONTACTS_FILE, "r", encoding="utf-8") as f:
            return json.load(f)
    return []

def save_contacts(contacts):
    with open(CONTACTS_FILE, "w", encoding="utf-8") as f:
        json.dump(contacts, f, indent=2)

def add_contact(name: str, phone: str, email: str):
    contacts = load_contacts()
    contacts.append({"name": name, "phone": phone, "email": email})
    save_contacts(contacts)
    print(f"Added: {name}")

def export_to_csv(filename="contacts.csv"):
    contacts = load_contacts()
    with open(filename, "w", newline="", encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=["name", "phone", "email"])
        writer.writeheader()
        writer.writerows(contacts)
    print(f"Exported to {filename}")

def show_contacts():
    contacts = load_contacts()
    if not contacts:
        print("No contacts yet.")
        return
    for i, c in enumerate(contacts, 1):
        print(f"{i}. {c['name']} | {c['phone']} | {c['email']}")

add_contact("Alice", "123-456-7890", "alice@email.com")
add_contact("Bob", "987-654-3210", "bob@email.com")
show_contacts()
export_to_csv()
```

Practice: Add search-by-name and delete functions. Store training-run metadata the same way.

6. Error Handling and Custom Exceptions

The basic idea

When something goes wrong (file missing, bad number, network timeout), Python raises an **exception** and stops the program. You can catch exceptions with `try / except` so your program can handle the problem instead of crashing.

You can also create your own exception types by inheriting from `Exception`. Custom exceptions make error messages clearer and help other developers (and future you) understand what went wrong.

Why this matters for AI

Real data is messy. Models fail. APIs time out. Robust pipelines catch specific errors, log them, and continue or fail gracefully.

Complete sample

```
# 06_safe_reader.py
class FileEmptyError(Exception):
    """Raised when a file is empty."""
    pass

class InvalidFormatError(Exception):
    """Raised when file content is not in expected format."""
    pass

def safe_read_numbers(filepath: str) -> list[float]:
    try:
        with open(filepath, "r") as f:
            content = f.read().strip()
        if not content:
            raise FileEmptyError(f"File '{filepath}' is empty")
        numbers = []
        for line in content.splitlines():
            line = line.strip()
            if not line:
                continue
        try:

```

```
        numbers.append(float(line))
    except ValueError:
        raise InvalidFormatError(f"Invalid number found: '{line}'")
    return numbers
except FileNotFoundError:
    print(f"Error: File '{filepath}' does not exist.")
    return []
except (FileEmptyError, InvalidFormatError) as e:
    print(f"Custom Error: {e}")
    return []
except Exception as e:
    print(f"Unexpected error: {e}")
    return []

with open("numbers.txt", "w") as f:
    f.write("10\n20\n3.5\nabc\n40\n")
print(safe_read_numbers("numbers.txt"))
print(safe_read_numbers("nonexistent.txt"))
```

Practice: Load a JSON config and raise a custom ConfigError if required keys are missing.

7. OOP and Dataclasses — Dataset Class

The basic idea

Object-Oriented Programming (OOP) lets you group data and the functions that work on that data into a **class**. A class is a blueprint; an **object** (or instance) is one concrete thing built from that blueprint.

Example: a Dataset class can store samples and also know how to compute the mean of a feature. You write the logic once inside the class and reuse it for every dataset.

A **dataclass** is a shortcut. With the `@dataclass` decorator, Python automatically creates the `__init__` method and a nice printable representation for you.

Why this matters for AI

PyTorch Dataset, scikit-learn estimators, and almost every modern library are built with classes. Understanding classes is required to use and extend them.

Complete sample

```
# 07_dataset_class.py
from dataclasses import dataclass, field
from typing import List, Any
import statistics

@dataclass
class Sample:
    features: List[float]
    label: Any = None

@dataclass
class Dataset:
    name: str
    samples: List[Sample] = field(default_factory=list)

    def add_sample(self, features: List[float], label=None):
        self.samples.append(Sample(features, label))

    def __len__(self):
        return len(self.samples)

    def mean_feature(self, index: int) -> float:
        values = [s.features[index] for s in self.samples]
        return statistics.mean(values)

    def summary(self):
        print(f"Dataset: {self.name}")
        print(f"Number of samples: {len(self)}")
        if self.samples:
            n_features = len(self.samples[0].features)
            print(f"Features per sample: {n_features}")
            for i in range(n_features):
                print(f"  Feature {i} mean: {self.mean_feature(i):.2f}")

ds = Dataset(name="Iris-like")
ds.add_sample([5.1, 3.5, 1.4, 0.2], label="setosa")
ds.add_sample([7.0, 3.2, 4.7, 1.4], label="versicolor")
ds.add_sample([6.3, 3.3, 6.0, 2.5], label="virginica")
ds.summary()
print(f"Length: {len(ds)}")

Practice: Add a train_test_split(self, ratio=0.8) method that returns two new Dataset objects.
```

8. Modules, Packages, venv, pip / uv

The basic idea

A **module** is a single Python file you can import. A **package** is a folder of modules (it needs an `__init__.py` file).

A **virtual environment** (venv) is an isolated Python installation for one project. It prevents library version conflicts between projects. **pip** is the tool that installs packages. **uv** is a modern, much faster alternative.

Why this matters for AI

You will install dozens of libraries (numpy, pandas, torch, transformers...). Virtual environments keep each project stable and reproducible.

Commands + sample

```
# Terminal commands
python -m venv .venv
# Windows: .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install --upgrade pip
pip install requests pandas
pip freeze > requirements.txt

# Simple reusable function (imagine this is inside myutils/math_helpers.py)
def clamp(value: float, min_val: float, max_val: float) -> float:
    return max(min_val, min(value, max_val))

print(clamp(15, 0, 10)) # → 10
print(clamp(-5, 0, 10)) # → 0

# With uv (faster):
# uv venv && source .venv/bin/activate
# uv pip install requests pandas

Practice: Create a small package called aiutils with a seed function and a timer decorator. Install it with pip install -e .
```

9. Reproducibility: Seeds and Pinned Environments

The basic idea

Many algorithms use randomness (shuffling data, initializing weights, dropout). If you do not control that randomness, every run gives slightly different results and you cannot trust comparisons.

A **seed** is a starting number for the random number generator. Setting the same seed makes the sequence of random numbers the same every time. You must set the seed for Python's random module, NumPy, and later PyTorch.

Pinning package versions (`pip freeze > requirements.txt`) ensures that another machine (or you six months later) installs exactly the same library versions.

Why this matters for AI

If you cannot reproduce a result, you cannot trust it or debug it. Reproducibility is a core scientific habit.

Complete sample

```
# 09_reproducibility.py
import random
import numpy as np

def set_seed(seed: int = 42):
    random.seed(seed)
    np.random.seed(seed)
    # Later for PyTorch:
    # import torch
    # torch.manual_seed(seed)
    # if torch.cuda.is_available():
    #     torch.cuda.manual_seed_all(seed)

set_seed(42)
print(random.random())
print(np.random.rand(3))
# Run this file twice – the numbers will be identical.

Practice: Extend set_seed to also set PYTHONHASHSEED and torch.backends.cudnn.deterministic = True.
```

10. Type Hints with mypy

The basic idea

Python does not require you to declare types, but you can optionally add **type hints**: `def average(numbers: List[float]) -> float: .` They document what the function expects and returns.

mypy is a tool that reads your type hints and checks for mistakes before you run the program (for example, passing a string where a list of floats is expected).

Why this matters for AI

Type hints catch entire classes of bugs early. Libraries like PyTorch, Pydantic, and FastAPI rely on them heavily.

Complete sample

```
# 10_type_hints.py
from typing import List, Dict, Optional, Union

def average(numbers: List[float]) -> float:
    if not numbers:
        raise ValueError("List cannot be empty")
    return sum(numbers) / len(numbers)

def find_user(users: Dict[str, dict], user_id: str) -> Optional[dict]:
    return users.get(user_id)

def process(value: Union[int, str]) -> str:
    return str(value).upper()

print(average([10.0, 20.0, 30.0]))
print(process(42))
# pip install mypy
# mypy 10_type_hints.py
```

Practice: Add type hints to the Dataset class from topic 7, then run mypy and fix any issues.

11. Pydantic: Validation and Settings

The basic idea

When data comes from a user, a file, or an API, it may be wrong (missing fields, wrong types, out-of-range values). **Pydantic** lets you declare a schema as a class. It then automatically validates and converts incoming data, and raises clear errors when something is invalid.

You can also use Pydantic to load settings from environment variables or a `.env` file. This is the standard way modern Python apps handle configuration.

Why this matters for AI

Pydantic powers FastAPI, LangChain, many LLM tools, and config systems. Validating training configs and API payloads early saves hours of debugging.

Complete sample

```
# 11_pydantic_demo.py
from pydantic import BaseModel, Field, EmailStr, field_validator

class User(BaseModel):
    id: int
    name: str = Field(..., min_length=2, max_length=50)
    email: EmailStr
    age: int = Field(..., ge=0, le=120)

    @field_validator("name")
    @classmethod
    def name_must_be_capitalized(cls, v: str) -> str:
        if not v[0].isupper():
            raise ValueError("Name must start with capital letter")
        return v

user = User(id=1, name="Alice", email="alice@example.com", age=28)
print(user.model_dump())
# pip install pydantic email-validator
# This will raise ValidationError:
```

```
# User(id=1, name="bob", email="bad", age=150)
```

Practice: Create a `TrainingConfig` model with `learning_rate`, `batch_size`, `epochs`, `model_name`. Load it from a dict or JSON.

12. Generators, Iterators, Decorators

The basic idea

A normal function builds the whole result and returns it. A **generator** yields one value at a time and pauses. This saves memory when the full result would be huge (for example, reading a multi-gigabyte file line by line).

You create a generator with the `yield` keyword instead of `return`.

A **decorator** is a function that wraps another function to add extra behavior (timing, logging, retry) without changing the original function's code. You apply it with the `@decorator_name` syntax.

Why this matters for AI

Generators let you stream large datasets. Decorators are how you add timing, logging, and retry logic cleanly.

Complete sample

```
# 12_generators_decorators.py
import time
from functools import wraps

def countdown(n: int):
    while n > 0:
        yield n
        n -= 1
print(list(countdown(5)))

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        result = func(*args, **kwargs)
        end = time.perf_counter()
        print(f"{func.__name__} took {end - start:.4f} seconds")
        return result
    return wrapper

@timer
def slow_function():
    time.sleep(0.3)
    return "Done"
print(slow_function())
```

Practice: Write a retry decorator (max 3 attempts). Write a generator that yields batches of a list.

13. Context Managers (with statement)

The basic idea

A **context manager** guarantees that setup and cleanup always happen, even if an error occurs in the middle. The most common example is opening a file:

`with open("data.txt") as f:` — the file is closed automatically when the block ends.

You can write your own context managers in two ways: a class with `__enter__` and `__exit__` methods, or a generator decorated with `@contextmanager`.

Why this matters for AI

Context managers are used for files, GPU memory, database connections, locks, and temporary directories. They prevent resource leaks in long-running training jobs.

Complete sample

```
# 13_context_managers.py
from contextlib import contextmanager
import time

class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc_val, exc_tb):
        self.elapsed = time.perf_counter() - self.start
        print(f"Elapsed: {self.elapsed:.4f}s")
        return False

with Timer():
    time.sleep(0.2)
    print("Working...")

@contextmanager
def open_and_count(path: str):
    f = open(path, "r", encoding="utf-8")
    try:
        lines = f.readlines()
        yield lines
    finally:
        f.close()
        print(f"Closed {path} ({len(lines)} lines)")

with open("demo_lines.txt", "w") as f:
    f.write("line one\nline two\nline three\n")
with open_and_count("demo_lines.txt") as lines:
    print("Content:", [l.strip() for l in lines])
```

Practice: Write a context manager that temporarily changes the working directory and restores it afterward.

14. collections (defaultdict, deque, NamedTuple)

The basic idea

The `collections` module adds useful container types on top of list/dict/set:

- **defaultdict** — a dict that automatically creates a default value for missing keys (no more `KeyError` checks).
- **deque** — a double-ended queue. Adding/removing from both ends is fast. Perfect for sliding windows.
- **NamedTuple** — like a lightweight immutable class. You get named fields and tuple behavior.
- **Counter** — counts occurrences (already used in the word counter).

Why this matters for AI

These structures appear daily in data pipelines, simple models, and preprocessing code.

Complete sample

```
# 14_collections_demo.py
from collections import defaultdict, deque, namedtuple, Counter

grouped = defaultdict(list)
for cat, item in [("fruit", "apple"), ("fruit", "banana"), ("veg", "carrot")]:
    grouped[cat].append(item)
```

```
print(dict(grouped))

window = deque(maxlen=3)
for x in [10, 20, 30, 40, 50]:
    window.append(x)
    print(list(window))

Sample = namedtuple("Sample", ["features", "label"])
s = Sample(features=[1.2, 3.4], label="positive")
print(s.features, s.label)

print(Counter("abracadabra").most_common(3))
```

Practice: Implement a moving-average calculator with `deque(maxlen=n)`. Use `defaultdict(int)` to count word-pair co-occurrences.

15. requests and REST APIs

The basic idea

A **REST API** is a way for programs to talk over the internet using HTTP (the same protocol browsers use). You send a request (GET, POST, etc.) to a URL and receive a response, often in JSON format.

The `requests` library makes this simple in Python. You check the status code (200 means success) and parse the JSON body.

Why this matters for AI

You will download datasets, call model APIs (OpenAI, Anthropic, Hugging Face), and fetch web data. Understanding status codes, headers, and JSON is essential.

Complete sample

```
# 15_api_demo.py
import requests

def get_github_user(username: str) -> dict:
    url = f"https://api.github.com/users/{username}"
    response = requests.get(url, timeout=10)
    if response.status_code == 200:
        return response.json()
    print(f"Error {response.status_code}: {response.text[:200]}")
    return {}

user = get_github_user("octocat")
if user:
    print(f"Name: {user.get('name')}")
    print(f"Public repos: {user.get('public_repos')}")
    print(f"Followers: {user.get('followers')}")
# pip install requests
```

Practice: Call the Hugging Face model hub API and print the first few results. Handle 429 rate-limit errors.

16. Async with asyncio and httpx

The basic idea

Normal (synchronous) code waits for each network request to finish before starting the next one. If each request takes 1 second, 10 requests take about 10 seconds.

Async code can start many requests at once and wait for all of them together. The same 10 requests might finish in a little more than 1 second. `asyncio` is Python's built-in async framework. `httpx` is an HTTP library that supports async.

You mark async functions with `async def` and wait for them with `await`. `asyncio.gather` runs many tasks concurrently.

Why this matters for AI

Calling many LLM APIs or downloading thousands of images is painfully slow if done one-by-one. Async keeps production RAG and agent systems fast.

Complete sample

```
# 16_async_demo.py
import asyncio
import httpx

async def fetch_url(client: httpx.AsyncClient, url: str) -> str:
    response = await client.get(url, timeout=10)
    return f"{url} → {response.status_code}"

async def main():
    urls = [
        "https://httpbin.org/delay/1",
        "https://httpbin.org/delay/1",
        "https://httpbin.org/delay/1",
        "https://api.github.com",
    ]
    async with httpx.AsyncClient() as client:
        tasks = [fetch_url(client, url) for url in urls]
        results = await asyncio.gather(*tasks)
    for r in results:
        print(r)

if __name__ == "__main__":
    asyncio.run(main())
# pip install httpx
```

***Practice:** Compare total time of this async version versus a pure requests version for the same URLs.*

17. Simple CLI with argparse

The basic idea

A **CLI** (Command-Line Interface) lets users pass options when they run a script, for example:

```
python train.py --epochs 50 --lr 0.001 --data ./data
```

The built-in `argparse` module parses these options, converts types (string → int/float/Path), sets defaults, and prints a help message automatically when the user passes `--help`.

Why this matters for AI

Almost every training or inference script needs command-line arguments (`--epochs`, `--batch-size`, `--data-path`, `--lr`). Learning `argparse` once pays off for years.

Complete sample

```
# 17_train_cli.py
import argparse
from pathlib import Path

def main():
    parser = argparse.ArgumentParser(description="Tiny training script example")
    parser.add_argument("--data", type=Path, required=True, help="Path to dataset")
    parser.add_argument("--epochs", type=int, default=10, help="Number of epochs")
    parser.add_argument("--batch-size", type=int, default=32, help="Batch size")
    parser.add_argument("--lr", type=float, default=1e-3, help="Learning rate")
    parser.add_argument("--output", type=Path, default=Path("artifacts"), help="Output dir")
    parser.add_argument("--verbose", action="store_true", help="Print extra logs")
```

```
args = parser.parse_args()

print("=== Training Config ===")
print(f>Data path      : {args.data}")
print(f>Epochs       : {args.epochs}")
print(f>Batch size    : {args.batch_size}")
print(f>Learning rate: {args.lr}")
print(f>Output dir    : {args.output}")
print(f>Verbose       : {args.verbose}")
if args.verbose:
    print("Starting fake training loop...")
print("Done.")

if __name__ == "__main__":
    main()

# python 17_train_cli.py --data ./data/train.csv
# python 17_train_cli.py --data ./data --epochs 50 --lr 0.0005 --verbose

Practice: Add a --seed argument and call set_seed from topic 9. Add --optimizer with choices (sgd, adam, adamw).
```

How to Practice Effectively

1. Create a folder and put every sample in its own file.
2. Run the sample exactly as written. Confirm the output matches the comments.
3. Break it on purpose (wrong types, missing files, bad keys). Watch the errors. Then fix them.
4. Complete the Practice challenge before moving to the next topic.
5. After all 17 topics, build one small combined project that uses at least 8 of them (example: a CLI that fetches data from an API, validates it with Pydantic, saves JSON, and prints stats).

Next Level

Once you are comfortable with Level 1, move to **Level 2: Math and Data Tools** (NumPy, Pandas, Matplotlib, basic statistics and linear algebra). That level gives you the numerical foundation required for every machine-learning algorithm that follows.

Python AI Bootcamp • Level 1: Python Foundations
Author: Venkatesan Vetrimurasu | YouTube: Venkat's Tech Lab
Keep coding. Keep breaking things. Keep fixing them.